NYC TAXI FLEET ANALYTICS

Azure Data Engineering — Complete Project Documentation

End-to-End Guide: Architecture · Code · Setup · Results

Azure Data Factory · Databricks PySpark · Delta Lake · Unity Catalog · Power BI

745,796 Records · $13.5M Revenue · 42 Delta Versions · 8 Consecutive Successful Runs

Author	Thadshayani Rasanehru
Role	Senior Data Engineer
Platform	Microsoft Azure
Cluster	Single node · Standard_DC4as_v5 · DBR 13.3 LTS
Date	March 2026
Status	Complete — Running in Production

Table of Contents

- Section 1 — Project Overview & Business Problem
- Section 2 — Complete File & Folder Structure
- Section 3 — Azure Infrastructure Setup
- Section 4 — Data Sources (API + Reference Files)
- Section 5 — Bronze Layer — ADF Pipeline
- Section 6 — Silver Layer — PySpark Transformations
- Section 7 — Gold Layer — Delta Lake Star Schema
- Section 8 — Incremental Loading & Watermark Strategy
- Section 9 — Audit Log & Error Handling
- Section 10 — Databricks Workflow — Scheduling & Automation
- Section 11 — Power BI Dashboard — DirectQuery
- Section 12 — Key Technical Challenges Solved
- Section 13 — Pipeline Metrics & Run History
- Section 14 — Business Insights from Dashboard
- Section 15 — Production Improvement Roadmap

SECTION 1 PROJECT OVERVIEW & BUSINESS PROBLEM

1.1 What This Project Does

This project builds a complete end-to-end Azure data engineering pipeline for a NYC green taxi fleet owner. It ingests 787,060 raw trip records from the NYC Taxi & Limousine Commission (TLC) API across 12 months of 2023, transforms them through a medallion architecture (Bronze → Silver → Gold), and delivers a live Power BI dashboard showing $13.5M in revenue analysis.

1.2 The Business Problem

The fleet owner had no consolidated reporting. Monthly trip files came from two dispatch vendors (VeriFone and Creative Mobile Technologies) with inconsistent schemas. There was no data quality filtering — zero-fare trips, duplicates, and invalid records were silently included in totals. No audit trail existed to debug wrong numbers. No geographic analysis was possible because location IDs appeared as raw integers.

1.3 Technology Stack

Cloud Platform	Microsoft Azure
Ingestion	Azure Data Factory (ADF) — Dynamic ForEach pipeline
Storage	Azure Data Lake Storage Gen2 (Bronze / Silver / Gold)
Processing	Azure Databricks — DBR 13.3 LTS
Language	PySpark / Python
Table Format	Delta Lake (ACID + Time Travel + MERGE)
Governance	Unity Catalog + External Locations
Orchestration	Databricks Workflows — 4 tasks, daily schedule
Visualization	Power BI Desktop — DirectQuery live connection
Security	Service Principal + OAuth2 + Managed Identity

SECTION 2 COMPLETE FILE & FOLDER STRUCTURE

The GitHub repository contains every file needed to understand, reproduce, and extend this project. Below is the complete structure with explanation of every file.

notebooks/nb_silver_cleanse_deliveries.ipynb	Main transformation notebook. Reads Bronze parquet, applies all 13 PySpark transformations, writes partitioned Silver parquet. Largest notebook at 153KB.
notebooks/nb_gold_delta_tables.ipynb	Creates all 3 Gold Delta tables. Reads Silver parquet for fact_deliveries. Reads taxi_zone_lookup.xlsx for dim_trip_zone. Reads trip_type.xlsx for dim_trip_type. Runs OPTIMIZE, ZORDER, VACUUM.
notebooks/nb_incremental_watermark.ipynb	Implements watermark-based incremental loading. Detects new Bronze files, processes only new ones, updates watermark table after each successful run.
notebooks/nb_merge_incremental.ipynb	Executes Delta MERGE on fact_deliveries. Matches on 6-column business key. INSERT new records, UPDATE changed records. No full overwrite.
notebooks/nb_audit_log.ipynb	Writes one audit record per pipeline run to audit_log Delta table. Records pipeline_name, status, records_read, records_written, records_rejected, duration_seconds, error_message.
adf_pipeline/pipeline/pl_ingest_delivery_records.json	Main ADF pipeline definition. Contains ForEach activity, Copy activity, parameterized source and sink paths.
adf_pipeline/dataset/ds_adls_delivery_raw_sink.json	ADF dataset definition for ADLS Gen2 sink (Bronze container).
adf_pipeline/dataset/ds_http_delivery_src.json	ADF dataset definition for NYC TLC HTTP source API.
adf_pipeline/linkedService/ls_adls_logistics_store.json	ADF linked service for ADLS Gen2 storage account.
adf_pipeline/linkedService/ls_http_logistics_api.json	ADF linked service for NYC TLC HTTP API endpoint.
databricks_workflow/logistics_pipeline_workflow.json	Databricks Workflow job definition. Contains all 4 task definitions, dependencies, schedule (06:00 AM Asia/Dubai), cluster config.

raw_data/taxi_zone_lookup.xlsx	Official NYC TLC zone reference file. 265 rows. Maps LocationID integers to Borough and Zone names. Used to build dim_trip_zone Gold table.
raw_data/trip_type.xlsx	Trip type reference file. 2 rows. Maps 1=Street Hail, 2=Dispatch. Used to build dim_trip_type Gold table.
powerbi/nyc_taxi_analytics_dashboard.pbix	Power BI dashboard file. Connects to Databricks Gold layer via DirectQuery. Contains all 6 visualizations and month filter slider.
docs/nyc_taxi_project_documentation.docx	This document. Complete project documentation.
screenshots/	Folder containing all 10 project screenshots for README and portfolio.

SECTION 3 AZURE INFRASTRUCTURE SETUP

3.1 Azure Resources Created

Resource Group	rg-logistics-analytics-prod — contains all project resources
Storage Account	adlslogisticsstore — ADLS Gen2 with hierarchical namespace enabled
ADLS Containers	bronze · silver · gold · raw-data — four separate containers
Databricks Workspace	adb-logistics-analytics — Unity Catalog enabled
Data Factory	adf-logistics-analytics — linked to ADLS via service principal
SQL Warehouse	logistics-sql-warehouse — Serverless XS — for Power BI DirectQuery

3.2 Service Principal & Authentication

All Databricks notebooks authenticate to ADLS Gen2 using a service principal via OAuth2. The service principal requires Storage Blob Data Contributor role on the storage account.

```
# Replace these values in all 5 notebooks
client_id     = 'your-service-principal-client-id'
tenant_id     = 'your-tenant-id'
client_secret = 'your-client-secret'

spark.conf.set('fs.azure.account.auth.type.<storage>.dfs.core.windows.net', 'OAuth')
spark.conf.set('fs.azure.account.oauth.provider.type.<storage>.dfs.core.windows.net'
,
               'org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider')
spark.conf.set('fs.azure.account.oauth2.client.id.<storage>.dfs.core.windows.net',
client_id)
spark.conf.set('fs.azure.account.oauth2.client.secret.<storage>.dfs.core.windows.net
', client_secret)
spark.conf.set('fs.azure.account.oauth2.client.endpoint.<storage>.dfs.core.windows.n
et',
               f'https://login.microsoftonline.com/{tenant_id}/oauth2/token')
```

3.3 Unity Catalog External Location Setup

When Unity Catalog is enabled, all ADLS write paths must be registered as External Locations. Without this, all writes fail with NO_PARENT_EXTERNAL_LOCATION_FOR_PATH.

Step 1	Locate unity-catalog-access-connector managed identity in Azure portal (created automatically when Unity Catalog was enabled)

Step 2	Create storage credential in Databricks: CREATE STORAGE CREDENTIAL sp_logistics_credential WITH AZURE MANAGED IDENTITY (CONNECTOR = 'unity-catalog-access-connector')
Step 3	Register Gold external location: CREATE EXTERNAL LOCATION gold_external_location URL 'abfss://gold@adlslogisticsstore.dfs.core.windows.net/' WITH (STORAGE CREDENTIAL sp_logistics_credential)
Step 4	Register Silver external location: same pattern pointing to silver container
Step 5	Validate: VALIDATE STORAGE CREDENTIAL sp_logistics_credential ON LOCATION 'abfss://gold@...' WITH READ, WRITE, DELETE, LIST

SECTION 4 DATA SOURCES

4.1 Source 1 — NYC TLC Trip Transaction Files (API)

Monthly parquet files published by the NYC Taxi and Limousine Commission. Downloaded via HTTP and ingested to Bronze by Azure Data Factory.

Files	green_tripdata_2023-01.parquet through green_tripdata_2023-12.parquet
Total files	12 monthly parquet files
Raw records	787,060 trip records before any filtering
Source URL	https://d37ci6vzurychx.cloudfront.net/trip-data/
Ingestion	Automated via ADF ForEach pipeline — zero manual steps
Bronze path	abfss://bronze@adlslogisticsstore.dfs.core.windows.net/delivery_records_2023/trip-data/

4.2 Key Columns in Trip Data

VendorID	1=VeriFone, 2=Creative Mobile Technologies — dispatch vendor
lpep_pickup_datetime	Trip start timestamp — source for all derived date/time columns
lpep_dropoff_datetime	Trip end timestamp — used for trip_duration_mins calculation
PULocationID	Pickup TLC zone ID — joins to dim_trip_zone.LocationID
DOLocationID	Dropoff TLC zone ID — joins to dim_trip_zone.LocationID
fare_amount	Metered fare — primary revenue metric. Records with 0 filtered out.
trip_distance	Miles driven. Records with 0 filtered out. Denominator in cost_per_mile.
payment_type	1=Credit Card, 2=Cash, 3=No Charge, 4=Dispute, 5=Unknown
trip_type	1=Street Hail, 2=Dispatch — joins to dim_trip_type
total_amount	Sum of all fare components — cross-validation reference

4.3 Source 2 — Dimension Reference Files

Two Excel files manually uploaded to ADLS Gen2 raw-data container. These are static reference files — they do not change frequently and do not need ADF pipeline processing.

taxi_zone_lookup.xlsx	265 rows. Official NYC TLC zone reference. Columns: LocationID, Borough, Zone, service_zone. Read by nb_gold_delta_tables.ipynb to create

	dim_trip_zone. Without this file, PULocationID and DOLocationID are meaningless integers in Power BI.
trip_type.xlsx	2 rows. Maps trip_type integer codes to descriptions. 1=Street Hail, 2=Dispatch. Read by nb_gold_delta_tables.ipynb to create dim_trip_type.
Upload path	abfss://raw-data@adlslogisticsstore.dfs.core.windows.net/

SECTION 5 BRONZE LAYER — ADF PIPELINE

5.1 Pipeline Design

The ADF pipeline pl_ingest_delivery_records uses a dynamic ForEach loop to ingest all 12 monthly parquet files from the NYC TLC HTTP endpoint to ADLS Gen2 Bronze container. Zero code changes are needed when new monthly files become available — only the loop parameter range changes.

Pipeline name	pl_ingest_delivery_records
Activity type	ForEach → Copy activity inside loop
Source dataset	ds_http_delivery_src — parameterized HTTP URL
Sink dataset	ds_adls_delivery_raw_sink — parameterized Bronze path
Linked services	ls_http_logistics_api (source) · ls_adls_logistics_store (sink)
Files ingested	12 monthly parquet files — Jan through Dec 2023
Records written	787,060 raw trip records
Transformation	None — Bronze stores raw data exactly as received

5.2 Why Bronze Stores Raw Data Untouched

- If Silver transformation logic changes (new column, new filter), the entire Silver layer can be rebuilt from Bronze without re-downloading from the TLC API
- Bronze is the source of truth — if a number is ever questioned, Bronze shows exactly what was received from the source
- Separating ingestion from transformation means ADF failures and Databricks failures are isolated and debugged independently

SECTION 6 SILVER LAYER — PYSPARK TRANSFORMATIONS

6.1 Schema Drift Resolution — Reading 12 Files Safely

The most critical step before any transformation. VendorID was stored as LONG in January but DOUBLE in subsequent months. Reading all 12 files together caused Photon engine to raise ClassCastException on month 8. The fix reads each file individually, casts all columns immediately, then unions all 12 DataFrames.

```
dfs = []
for month in range(1, 13):
    path = f'abfss://bronze@.../green_tripdata_2023-{month:02d}.parquet'
    df = spark.read.format('parquet').load(path)
    df = df.selectExpr(
        'CAST(VendorID AS DOUBLE) AS VendorID',
        'CAST(lpep_pickup_datetime AS TIMESTAMP) AS lpep_pickup_datetime',
        'CAST(fare_amount AS DOUBLE) AS fare_amount',
        # ... all 20 columns cast explicitly
    )
    dfs.append(df)

from functools import reduce
df_all = reduce(lambda a, b: a.unionByName(b), dfs)
df_all.cache().count()  # CRITICAL — force full materialisation into memory
# Bronze never touched again during this notebook execution
```

6.2 All 13 Transformation Steps

Step	Transform	Detail
1	Schema Drift Fix	Per-file selectExpr cast — VendorID LONG vs DOUBLE conflict resolved
2	Deduplication	dropDuplicates on 6-column key: VendorID, pickup_dt, dropoff_dt, PULocationID, DOLocationID, fare_amount — removed 35 duplicates
3	Column Rename	All columns standardised to lowercase snake_case
4	Invalid Filter	WHERE fare_amount > 0 AND trip_distance > 0 AND vendor_id IS NOT NULL
5	Date Extraction	trip_date, trip_year, trip_month derived from lpep_pickup_datetime
6	Year Filter	WHERE trip_year == 2023 — removes 2009/2022 meter clock reset anomalies
7	Trip Duration	(unix_timestamp(dropoff) - unix_timestamp(pickup)) / 60 → trip_duration_mins

8	**Suspicious Flag**	is_suspicious = duration<1 OR duration>300 OR fare>500 OR distance>100
9	**Payment Desc**	CASE: 1=Credit Card, 2=Cash, 3=No Charge, 4=Dispute, 5=Unknown
10	**Trip Type Desc**	CASE: 1=Street Hail, 2=Dispatch
11	**Cost Per Mile**	fare_amount / trip_distance — zero division protected by step 4
12	**Time of Day**	hour 6-11=Morning, 12-17=Afternoon, 18-21=Evening, else=Night
13	**Audit Columns**	ingestion_timestamp, source_system='NYC_TLC', pipeline_name added

6.3 Silver Write — Partitioned Output

Silver data is written partitioned by trip_year and trip_month. This produces 48 parquet files across 12 monthly subdirectories. The cache().count() before the write loop is critical — without it, Spark lazy evaluation re-triggers Bronze reads on each write iteration.

```
df_silver.cache()
df_silver.count()  # force ALL 745,796 records into cluster memory ONCE

for month in range(1, 13):
    df_month = df_silver.filter(col('trip_month') == month)
    df_month.write.mode('overwrite').parquet(
        f'abfss://silver@.../trip_year=2023/trip_month={month}/'
    )

# Result: 48 parquet files across 12 month partitions
# Silver path:
abfss://silver@adlslogisticsstore.dfs.core.windows.net/delivery_records/
```

SECTION 7 GOLD LAYER — DELTA LAKE STAR SCHEMA

7.1 Star Schema Design

The Gold layer implements a star schema with one fact table and two dimension tables, all registered in Unity Catalog under adb_logistics_analytics → logistics_db. Power BI's DAX engine is optimised for star schema — queries against this structure are significantly faster than against a denormalised flat table.

fact_deliveries	745,796 rows. Source: Silver parquet. Contains all numeric trip metrics, derived KPIs, and foreign keys to dimensions. Central query target for all revenue and volume analysis.
dim_trip_zone	265 rows. Source: raw_data/taxi_zone_lookup.xlsx. Maps TLC LocationID integers to Borough, Zone, service_zone. Enables geographic analysis in Power BI.
dim_trip_type	2 rows. Source: raw_data/trip_type.xlsx. Maps trip_type 1=Street Hail, 2=Dispatch. Enables trip origin segmentation.
audit_log	Per run. Tracks every pipeline execution with records read/written/rejected, status, duration, error messages.
watermark	1 row. Tracks last processed Bronze file for incremental loading. Updated after each successful run.

7.2 Delta Lake Features Applied

OPTIMIZE	Compacts 48 small Silver parquet files into fewer large Gold Delta files. Improves read performance.
ZORDER by trip_date, vendor_id	Sorts data within files by these columns. Power BI queries filtering by trip_date or vendor_id skip irrelevant files entirely — file-level pruning.
VACUUM (168 hours)	Removes Delta transaction log files older than 7 days. Controls storage cost. 168-hour retention keeps enough history for debugging.
Time Travel	42 Delta versions tracked. Any historical version queryable: SELECT * FROM fact_deliveries VERSION AS OF 0 returns the initial January-only load of ~64K records.
MERGE	INSERT new records + UPDATE changed records in a single atomic operation. No full table overwrite. No data loss window.
ACID Transactions	Every write is atomic. If a write fails mid-operation, the table remains in its previous valid state.

7.3 Loading Dimension Tables from Excel

```
# dim_trip_zone — from taxi_zone_lookup.xlsx
```

```
df_zones = spark.read.format('com.crealytics.spark.excel')
    .option('header', 'true')
    .load('abfss://raw-
data@adlslogisticsstore.dfs.core.windows.net/taxi_zone_lookup.xlsx')

df_zones.write.format('delta').mode('overwrite')
    .saveAsTable('logistics_db.dim_trip_zone')

# dim_trip_type — from trip_type.xlsx (same pattern)
# 2 rows: 1=Street Hail, 2=Dispatch
```

SECTION 8 INCREMENTAL LOADING & WATERMARK STRATEGY

8.1 Why Incremental Loading

Without incremental loading, the pipeline does a full reload every day — reading all 787,060 Bronze records, applying all 13 transformations, and overwriting all Gold records. The TLC API only adds one new file per month. Full reload wastes ~29 minutes of compute on 30 out of 31 days per month where nothing has changed.

8.2 Watermark Table Design

pipeline_name	Identifies which pipeline owns this watermark row
source_path	ADLS Bronze container path being monitored
last_file	Full filename of last successfully processed file e.g. green_tripdata_2023-12.parquet
last_processed	Date string of last processed file — used as comparison baseline
records_processed	Count from last run — sanity check baseline
processed_at	Timestamp of last watermark update

8.3 Three Run Scenarios

Run 1 — First Execution: No watermark record exists → full load all 12 files → 787,060 records processed → watermark written: last_file = green_tripdata_2023-12.parquet

Run 2 — No New Files: Watermark found: 2023-12 → all 12 Bronze files ≤ watermark → 0 files to process → pipeline exits cleanly in seconds → no compute wasted

Run 3 — New File Arrives: green_tripdata_2024-01.parquet detected → filename > watermark 2023-12 → only that 1 file processed → MERGE runs → watermark updated to 2024-01 → ~18 minutes total

8.4 Delta MERGE Logic

The MERGE operation matches on the same 6-column composite business key used for Silver deduplication. This ensures no duplicates can be introduced into Gold even if the same Silver data is reprocessed.

```
from delta.tables import DeltaTable

gold_table = DeltaTable.forName(spark, 'logistics_db.fact_deliveries')
```

```
gold_table.alias('gold').merge(
    df_silver.alias('silver'),
    '''gold.vendor_id = silver.vendor_id AND
       gold.lpep_pickup_datetime = silver.lpep_pickup_datetime AND
       gold.lpep_dropoff_datetime = silver.lpep_dropoff_datetime AND
       gold.pickup_location_id = silver.pickup_location_id AND
       gold.dropoff_location_id = silver.dropoff_location_id AND
       gold.fare_amount = silver.fare_amount'''
).whenMatchedUpdateAll()   # UPDATE if business key exists
 .whenNotMatchedInsertAll() # INSERT if new business key
 .execute()
```

SECTION 9 AUDIT LOG & ERROR HANDLING

9.1 Audit Log Table Schema

pipeline_name	ADF pipeline that triggered the run — pl_ingest_delivery_records
notebook_name	Specific Databricks notebook — enables per-notebook tracking
layer	bronze / silver / gold — filter by architecture layer
status	SUCCESS or FAILED — primary monitoring filter
records_read	Records read from source layer
records_written	Records written to target layer
records_rejected	records_read minus records_written — 41,264 for Silver (5.2%)
error_message	Full exception text first 200 characters for FAILED runs
start_time	Execution start timestamp
end_time	Execution end timestamp
duration_seconds	end_time minus start_time — Silver ~1745s, Gold ~265s, MERGE ~40s
run_date	Date string — primary filter for date-range audit queries

9.2 Error Handling Pattern

Each file operation is wrapped in an individual try/except block. A single corrupt or missing file does not terminate the pipeline — the error is caught, logged to the audit table, and the loop continues to the next file.

```
for file_path in files_to_process:
    try:
        df = spark.read.parquet(file_path)
        count = df.count()
        write_audit_log(notebook, 'silver', 'SUCCESS', count, 0, None)
        return count, None

    except Exception as e:
        error_msg = str(e)[:200]  # first 200 chars of error
        write_audit_log(notebook, 'silver', 'FAILED', 0, 0, error_msg)
        continue  # skip bad file — pipeline keeps going

# Tested:
# Valid file   → SUCCESS · 68,211 records returned
# Missing file → FAILED  · PATH_NOT_FOUND logged · pipeline continued
```

SECTION 10 DATABRICKS WORKFLOW — SCHEDULING & AUTOMATION

10.1 Workflow Configuration

Workflow name	logistics_pipeline
Job ID	551166306114141
Creator	Thadshayani Rasanehru
Schedule	06:00 AM UTC+04:00 — Asia/Dubai — daily
Cluster	Single node · Standard_DC4as_v5 · DBR Release 13.3.55
Task dependency	Run if All succeeded — cascading success gates

10.2 4-Task Pipeline

Task	Duration	Purpose
01_silver_cleanse	**~29 min**	Read 12 Bronze files, apply 13 PySpark transformations, write 48 Silver parquet files
02_gold_delta_tables	**~4 min 25s**	Create/refresh 3 Gold Delta tables. Load dims from Excel. OPTIMIZE + ZORDER + VACUUM.
03_merge_incremental	**~40 sec**	Delta MERGE Silver into fact_deliveries. INSERT new, UPDATE changed. No overwrite.
04_audit_log	**~9 sec**	Write one audit record capturing full run statistics for the pipeline execution.

10.3 Actual Run History — All Succeeded

Date	Duration	Launched By
Mar 03, 2026 06:00 AM	**18m 40s**	Scheduler
Mar 02, 2026 06:00 AM	**17m 17s**	Scheduler
Mar 01, 2026 06:00 AM	**18m 48s**	Scheduler
Feb 28, 2026 06:00 AM	**18m 16s**	Scheduler
Feb 27, 2026 06:00 AM	**18m 37s**	Scheduler
Feb 26, 2026 06:00 AM	**16m 42s**	Scheduler
Feb 25, 2026 06:00 AM	**18m 18s**	Scheduler
Feb 24, 2026 12:10 PM	**34m 41s**	Manual (initial full load)

SECTION 11 POWER BI DASHBOARD — DIRECTQUERY

11.1 Connection Setup

Connection type	DirectQuery — live connection, no data import, always current
Connector	Databricks connector in Power BI Desktop
Server hostname	adb-7405614196839780.0.azuredatabricks.net
HTTP path	From SQL Warehouse → Connection details → HTTP path
Authentication	Personal Access Token from Databricks user settings
Tables used	logistics_db.fact_deliveries · logistics_db.dim_trip_zone · logistics_db.dim_trip_type

11.2 6 Visualizations

KPI Card — Total Revenue	SUM(fact_deliveries[fare_amount]) = $13.5M
KPI Card — Total Trips	COUNT(fact_deliveries[trip_id]) = 745,796
KPI Card — Avg Cost/Mile	AVERAGE(fact_deliveries[cost_per_mile]) = $13.1
Monthly Revenue Bar Chart	fare_amount by trip_month — January through December — September peak $1.26M
Revenue by Payment Type Donut	fare_amount by payment_description — Credit Card 58.93%, Cash 30.78%, Unknown 10.01%
Trips by Time of Day Column Chart	COUNT trips by time_of_day — Afternoon 238K, Evening 199K, Morning 190K, Night 119K
Average Fare Trend Line	AVERAGE(fare_amount) by trip_month — Jan $16.5 → Sep $20.4 → Dec $18.5
Month Filter Slider	Range slider 1-12 — filters all 5 visuals simultaneously

SECTION 12 KEY TECHNICAL CHALLENGES SOLVED

Challenge 1 — Photon Engine Schema Conflict

Problem: VendorID stored as LONG in January but DOUBLE in other months. ClassCastException: MutableDouble cannot be cast to MutableLong on month 8 every run.

Root Cause: Reading all 12 files together in one spark.read.parquet() call caused Photon to perform cross-file schema unification and detect the type mismatch.

Solution: Read each file individually in a loop. Cast all columns via selectExpr immediately. Force materialisation with cache().count() before union. Photon never compares schemas across files.

Challenge 2 — Unity Catalog Blocking ADLS Writes

Problem: Workspace upgraded to Unity Catalog. All Silver and Gold writes failed: NO_PARENT_EXTERNAL_LOCATION_FOR_PATH. DBFS root access also disabled.

Solution: Located unity-catalog-access-connector managed identity → created sp_logistics_credential → registered gold_external_location and silver_external_location → validated Read, Write, Delete, List permissions.

Challenge 3 — Silver Write Loop Stopping at Month 1

Problem: Month-by-month write loop produced data only for month 1. Months 2-12 were empty folders with 0 bytes written.

Root Cause: Spark lazy evaluation — each write iteration re-triggered the entire Bronze read and selectExpr plan. From month 2 onward, the unresolved Photon schema conflict re-occurred.

Solution: df_silver.cache() followed by df_silver.count() before entering the write loop. This forces all 745,796 records into cluster memory. The write loop reads from cache — Bronze is never touched again.

SECTION 13 PIPELINE METRICS & RUN HISTORY

Metric	Value	Significance
Raw Records (Bronze)	**787,060**	12 monthly TLC files — complete 2023 dataset
Clean Records (Silver)	**745,796**	94.8% quality retention after all 13 transformations
Rejected Records	**41,264**	5.2% rejection — zero fare/distance, null vendor, wrong year
Duplicate Records Removed	**35**	6-column business key deduplication
Silver Parquet Files	**48**	4 files per month × 12 months — partitioned output
Gold Delta Tables	**3**	fact_deliveries + dim_trip_zone + dim_trip_type
Production Tables	**5**	Plus audit_log and watermark in logistics_db
Delta Versions	**42**	Full transaction history — all versions queryable
Gold Table Size	**21.2 MB**	Post-OPTIMIZE compressed Delta format
Total Revenue Analyzed	**$13.5M**	Sum of fare_amount across 745,796 clean trips
Initial Full Run	**34m 41s**	Feb 24 manual run — all 12 months processed
Daily Scheduled Runs	**~18 minutes**	Incremental — only new files processed
MERGE Performance	**40 seconds**	vs 29 min full rebuild — 97% compute reduction
Daily Schedule	**06:00 AM**	Asia/Dubai UTC+04:00 — pre-business-day completion
Consecutive Successful Runs	**8**	Feb 24 through Mar 03 — zero failures

SECTION 14 BUSINESS INSIGHTS FROM DASHBOARD

14.1 Revenue by Month

September — $1.26M (highest)	Convergence of extended tourist season, post-summer return-to-office commuter volume, and longer average trip distances. September also has the highest average fare at $20.4.
February — $1.02M (lowest)	Harsh winter weather suppresses discretionary travel. Shorter trips as residents limit journeys to essentials only.
January — $16.5 avg fare	Higher proportion of short inner-borough trips. Tourists absent. Commuters using subway more in cold weather.
September — $20.4 avg fare	+24% vs January. Longer tourist transfers, airport runs, outer-borough journeys. Congestion surcharge triggered more frequently in Manhattan zones.

14.2 Payment Behaviour

Credit Card — 58.93% ($7.97M)	TLC 2008 mandate established card infrastructure. Fully traceable, auto-reconciled, deposited in 1-2 days. tip_amount captured for all card transactions.
Cash — 30.78% ($4.16M)	Manual reconciliation required. tip_amount is NULL for all cash trips in TLC data — actual cash revenue is consistently understated in the dataset.
Unknown — 10.01% ($1.35M)	Payment method not recorded — likely connectivity failures during transaction processing by vendor terminal.

14.3 Trip Volume by Time of Day

Afternoon — 238K (busiest)	Hours 12-17. Lunch trips + school pickups + onset of evening commute overlap. 31.9% of annual volume.
Evening — 199K	Hours 18-21. Commute return + dining and entertainment. Second highest volume.
Morning — 190K	Hours 6-11. Outbound commute. Slightly lower than evening — more commuters use subway outbound.
Night — 119K (lowest volume)	Hours 22-5. Lowest trip count but highest average fare — airport runs, late-night entertainment, absence of short-hop daytime trips.

SECTION 15 PRODUCTION IMPROVEMENT ROADMAP

15.1 Security — Highest Priority

Current Gap: Service principal client_id, tenant_id, and client_secret are hardcoded as plain text in all 5 notebooks. Any user with notebook access can read these credentials.

Resolution: Create Azure Key Vault-backed secret scope in Databricks. Store all credentials as Key Vault secrets. Replace hardcoded strings with dbutils.secrets.get(scope='kv-logistics', key='sp-client-id'). Secrets retrieved via dbutils are redacted from all notebook output and Spark logs.

15.2 Architecture Improvements

- Multi-node auto-scaling cluster — current single node (4 cores, 16GB) processes 787K records in 29 minutes. At 5 years of data (~4M records) runtime exceeds 2.5 hours.
- Gold table partitioning — fact_deliveries has no partition columns. At 10M+ records, adding partition by trip_year/trip_month would enable partition pruning and significantly faster Power BI queries.
- Unity Catalog data lineage — currently disabled. Enabling it would allow tracing any Gold record back to the specific Bronze source file it originated from.

15.3 Data Quality Improvements

- Rejection rate threshold enforcement — pipeline currently succeeds even at 50% rejection. Add logic to fail pipeline and alert if rejection rate exceeds 10%.
- Cross-layer reconciliation — compare Silver record count against Bronze record count after each run. Silver count higher than Bronze would indicate a union bug.
- Dimension file version history — if taxi_zone_lookup.xlsx is updated, there is no audit trail of what changed. Dim tables should eventually use Delta MERGE with version history.

15.4 Operations Improvements

- Pipeline failure alerting — currently no email or Teams notification on failure. Team discovers failure only when Power BI users report stale data hours later.
- Automatic retry — transient Azure networking failures cause avoidable pipeline failures. Databricks Workflow supports 2 retries with 5-minute delay per task.
- CI/CD with Git integration — notebooks currently deployed manually by upload. No version control history, no automated testing before deployment.

PROJECT SUMMARY

This document covers the complete end-to-end Azure data engineering pipeline built for NYC green taxi fleet analytics. Using Azure Data Factory, Databricks PySpark, Delta Lake, Unity Catalog, and Power BI, the pipeline ingests 787,060 raw trip records from two source types (NYC TLC API + Excel reference files), transforms them through Bronze → Silver → Gold medallion architecture, and delivers a live $13.5M revenue dashboard via DirectQuery.

Any person reading this document should be able to understand every design decision made, reproduce the entire pipeline from scratch, and extend it with new data sources or transformations.

Author: Thadshayani Rasanehru · Senior Data Engineer · March 2026